

CGS698C - Assignment 2

Chaitanya Vishnoi (230316)

June 13, 2025

Part 1: A Simple Binomial Model

We are given:

- The data : $y = 7$
- The marginal likelihood :

$$\int L(\theta|y) \cdot p(\theta) d\theta = \frac{1}{11}$$

- The likelihood function :

$$L(\theta|y) = \binom{10}{y} \cdot \theta^y \cdot (1 - \theta)^{10-y}$$

- The prior assumption :

$$p(\theta) = \begin{cases} 1, & 0 \leq \theta \leq 1 \\ 0, & \text{otherwise} \end{cases}$$

1.1

Using the data to simplify the likelihood function:

$$L(\theta|7) = \binom{10}{7} \cdot \theta^7 \cdot (1 - \theta)^3 \Rightarrow 120 \cdot \theta^7 \cdot (1 - \theta)^3$$

$$\text{Using: } p(\theta|y) = \frac{L(\theta|y) \cdot p(\theta)}{\int L(\theta|y) \cdot p(\theta) d\theta}$$

We can see that

$$\Rightarrow p(\theta|7) = 1320 \cdot \theta^7 (1 - \theta)^3, \quad \text{for } 0 \leq \theta \leq 1 \quad \left(\text{Because, } \frac{120}{\frac{1}{11}} = 1320 \right)$$

Evaluations:

(a) posterior density for $\theta = 0.75$:

$$p(0.75|7) = 1320 \cdot 0.75^7 \cdot 0.25^3 \approx 2.7531$$

(b) posterior density for $\theta = 0.25$:

$$p(0.25|7) = 1320 \cdot 0.25^7 \cdot 0.75^3 \approx 0.0340$$

(c) posterior density for $\theta = 1$:

$$p(1|7) = 0$$

1.2 Posterior Distribution $p(\theta|y)$:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import binom

y = 7
N = 10
marginal_likelihood = 1/11

likelihoods = pd.DataFrame({'theta': np.arange(0, 1.0001, 0.0001)})

likelihoods['lkl'] = binom.pmf(y, N, likelihoods['theta'])

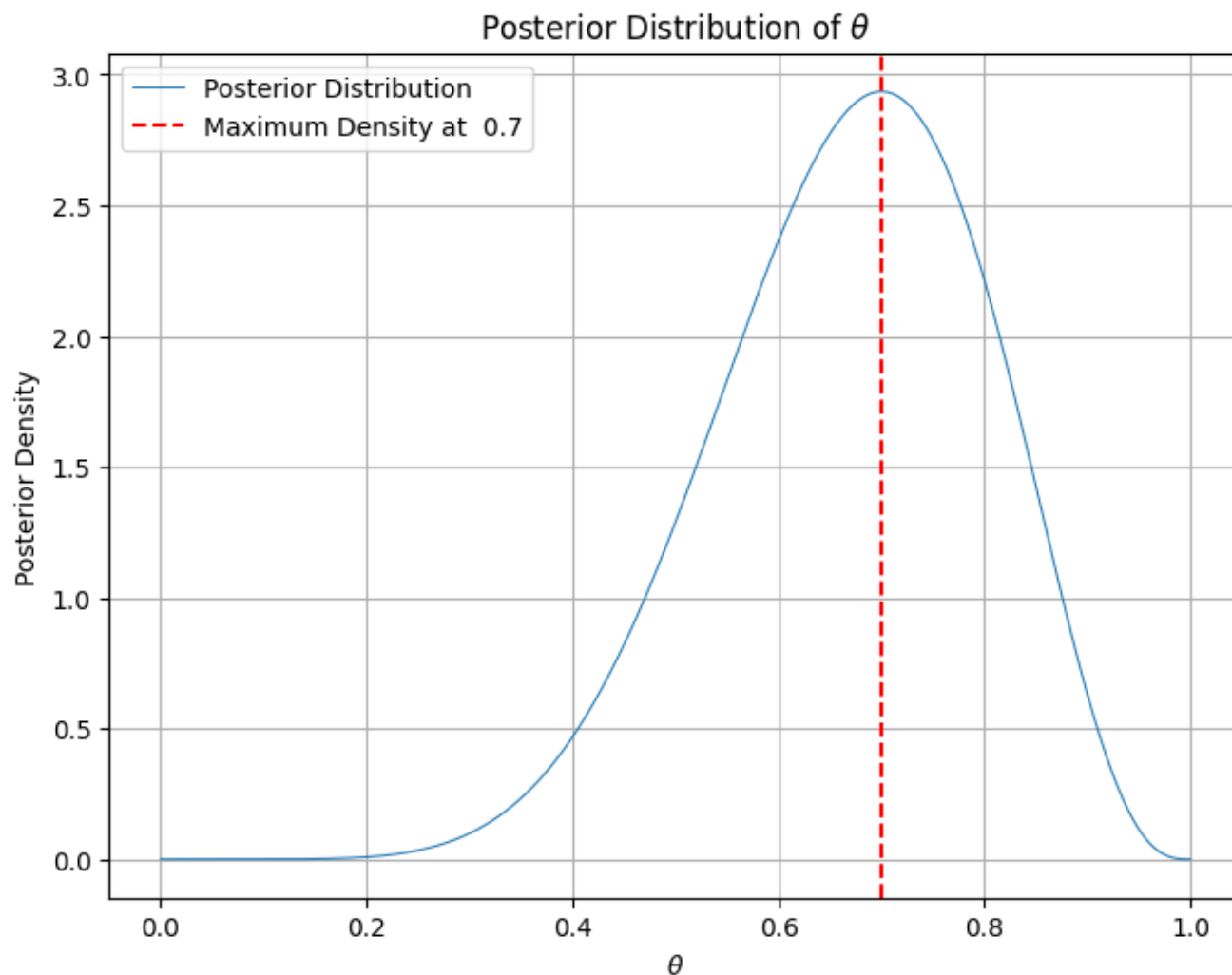
likelihoods['prior'] = 1

likelihoods['posterior_unnorm'] = likelihoods['lkl'] * likelihoods['prior']

likelihoods['posterior'] = likelihoods['posterior_unnorm'] /
    marginal_likelihood

max_density = np.max(likelihoods['posterior'])
max_theta = likelihoods['theta'][likelihoods['posterior'].idxmax()]

plt.figure(figsize=(4, 3))
plt.plot(likelihoods['theta'], likelihoods['posterior'], linewidth=0.75,
    label = 'Posterior_Distribution')
plt.xlabel(r'$\theta$')
plt.ylabel('Posterior_Density')
plt.title('Posterior_Distribution_of_' + r'$\theta$')
plt.axvline(x=max_theta, color='r', linestyle='--', label=f'Maximum_Density_
    at_{max_theta:.1f}')
plt.legend()
plt.grid(True)
plt.show()
```



1.3 Maximum posterior density:

Calculated using,

```
max_theta = likelihoods['theta'][likelihoods['posterior'].idxmax()]
```

$$\theta_{\text{MAX}} = 0.7$$

1.4 Graphs

```
import seaborn as sns
```

```
likelihoods.columns = ["theta", "likelihood", "prior", "posterior_unnorm", "
    posterior"]
likelihoods_m = pd.melt(likelihoods, id_vars=['theta'], var_name='variable',
    value_name='value')
```

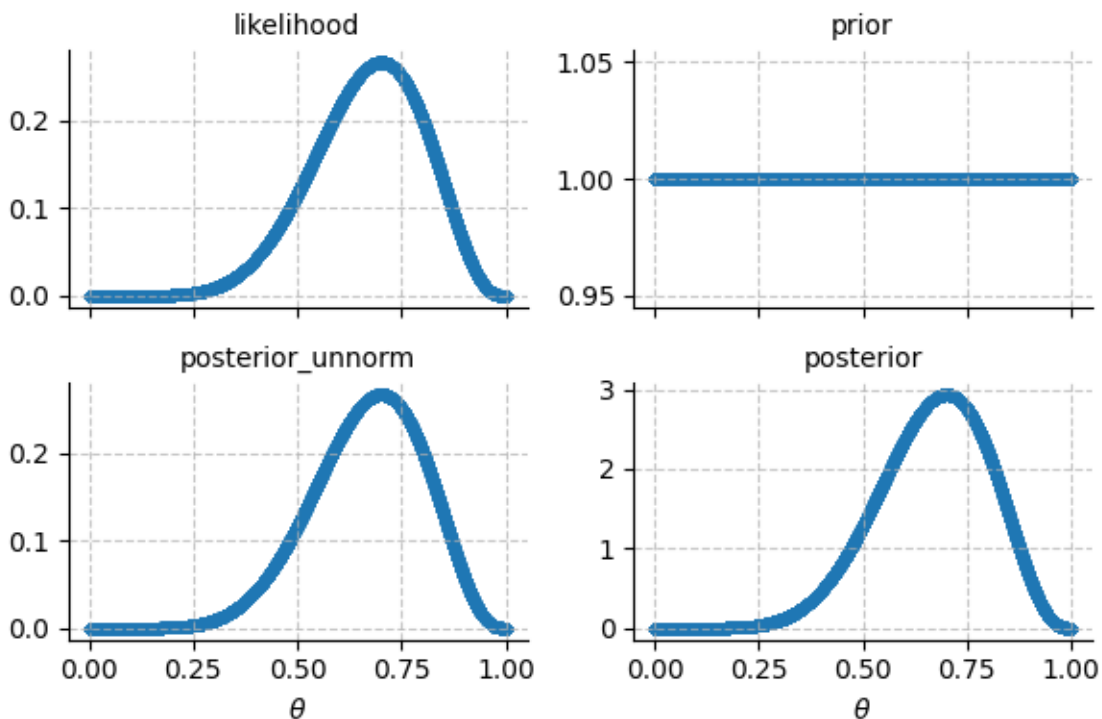
```

g = sns.FacetGrid(likelihoods_m, col="variable", col_wrap=2, sharey=False,
    height=2, aspect=1.5)
g.map(plt.scatter, "theta", "value", s=10)
g.set_axis_labels(r"$\theta$", "")
g.set_titles(col_template="{col_name}")

for ax in g.axes.flat:
    ax.grid(True, linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()

```



Part 2: A Gaussian Model of Reading

$$y = \{y_1, y_2, \dots, y_8\} = \{300, 270, 390, 450, 500, 290, 680, 450\}$$

$$y_i \sim \mathcal{N}(\mu, \sigma = 50), \quad \mu \sim \mathcal{N}(250, 25)$$

$$\mathcal{L}(\mu, \sigma | \mathbf{y}) = \frac{1}{(\sigma\sqrt{2\pi})^n} e^{-\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \mu)^2}$$

2.1 Unnormalized Posterior Density

To calculate, we use $p'(\mu|\sigma, y) = L(\mu, \sigma|y) \cdot p(\mu)$

```
import numpy as np

def like_est(y, mu, sigma):
    n = len(y)
    return (1/((sigma*np.sqrt(2*np.pi))**n))*np.exp(-(np.sum((y - mu)**2)))/(2*(sigma**2)))

def prior_est(mu):
    mu_p = 250
    sigma_p = 25
    return (1/(np.sqrt(2*np.pi*(sigma_p**2))))*np.exp(-((mu - mu_p)**2)/(2*(sigma_p**2)))

def post_est(y, mu, sigma):
    return like_est(y, mu, sigma)*prior_est(mu)

y = np.array([300.0, 270.0, 390.0, 450.0, 500.0, 290.0, 680.0, 450.0])
sigma = 50.0
mu_arr = [300.0, 900.0, 50.0]

for mu in mu_arr:
    print(f'Value of Unnormalized Posterior at (mean={mu:.2f}) is: ', \
          post_est(y, mu, sigma))
    print("\n")
}
```

Example evaluations:

(a) At $\mu = 300$, posterior $\approx 6.82e - 41$

(b) At $\mu = 900$, posterior = 0.0

(c) At $\mu = 50$, posterior $\approx 9.69e - 138$

2.2 Plotting Unnormalised Posterior Distribution $p'(\mu, \sigma|y)$

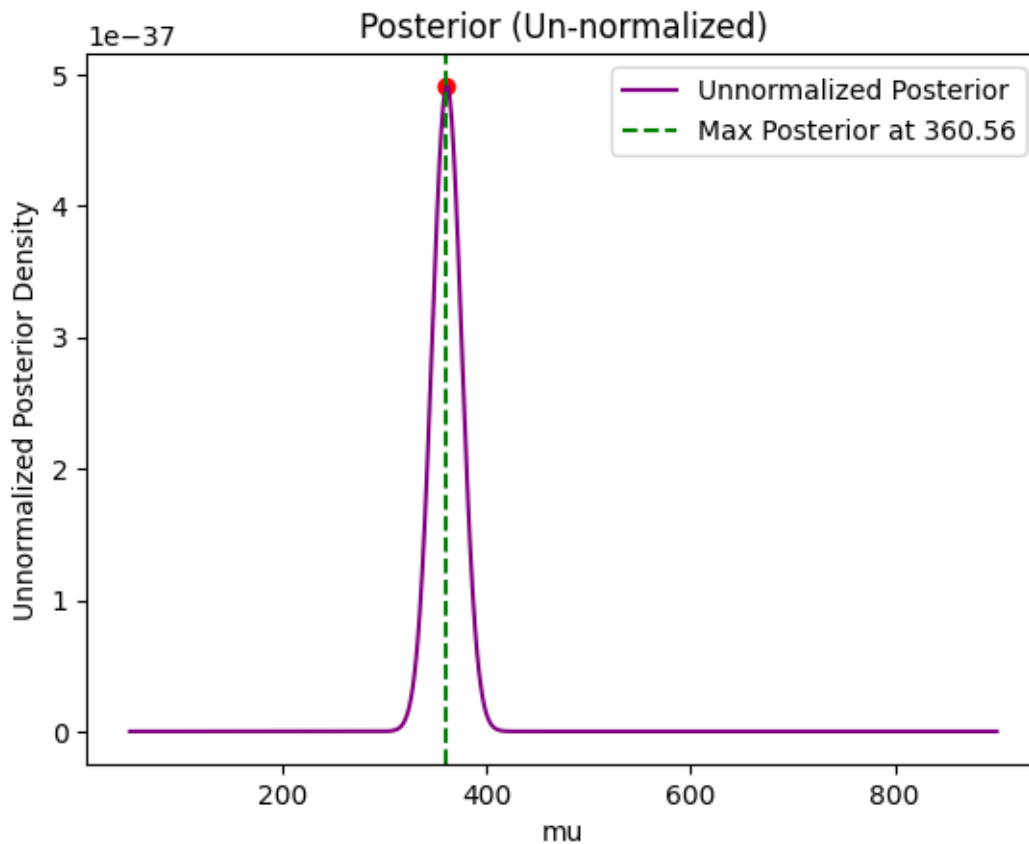
```
x_mu = np.linspace(50, 900, 1000)
y_post = [post_est(y, mu, sigma) for mu in x_mu]

max_idx = np.argmax(y_post)
max_mu = x_mu[max_idx]
max_post = y_post[max_idx]
```

```

plt.plot(x_mu, y_post, label='Unnormalized Posterior', linestyle='-', color=
'purple')
plt.axvline(max_mu, color='green', linestyle='--', label=f'Max Posterior at {
max_mu:.2f}')
plt.scatter(max_mu, max_post, color='red')
plt.xlabel('mu')
plt.ylabel('Unnormalized Posterior Density')
plt.title('Posterior (Un-normalized)')
plt.legend()
plt.show()
print("\n")

```



2.3 Combined Plot of prior and the (unnormalized) posterior distribution of μ

```

y_prior = [prior_est(mu) for mu in x_mu]

fig, axes = plt.subplots(1, 2, figsize=(7, 3))

axes[0].plot(x_mu, y_prior, label='Prior', linestyle='--', color='orange')

```

```

axes[0].plot(x_mu, y_post, label='Unnormalized Posterior', linestyle='--',
             color='purple')
axes[0].axvline(max_mu, color='green', linestyle='--', label=f'Max Posterior
               at {max_mu:.2f}')
axes[0].scatter(max_mu, max_post, color='red')
axes[0].set_xlabel('mu')
axes[0].set_ylabel('Probability Density')
axes[0].set_title('Prior and Unnormalized Posterior')
axes[0].legend()
axes[0].grid(True, linestyle='--', alpha=0.7)

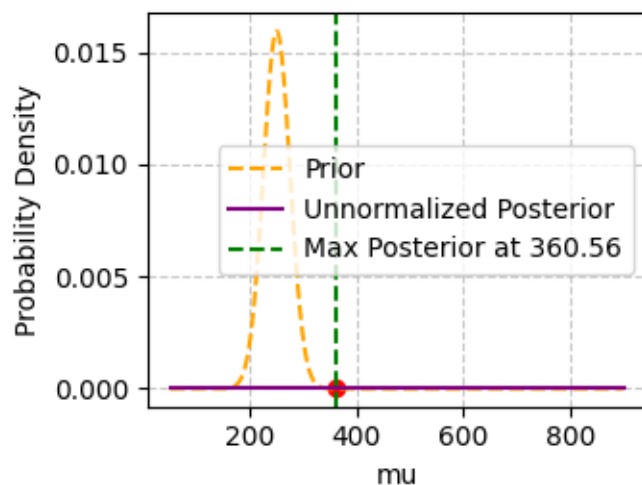
y_post_scaled = [(post_est(y, mu, sigma))*1e+34 for mu in x_mu]

axes[1].plot(x_mu, y_prior, label='Prior', linestyle='--', color='orange')
axes[1].plot(x_mu, y_post_scaled, label='Scaled Posterior (x 1e+34)',
             linestyle='--', color='purple')
axes[1].axvline(max_mu, color='green', linestyle='--', label=f'Max Posterior
               at {max_mu:.2f}')
axes[1].scatter(max_mu, max_post * 1e+34, color='red') # Scale max_post as
               well for the scatter plot
axes[1].set_xlabel('mu')
axes[1].set_title('Prior and Scaled Posterior')
axes[1].legend()
axes[1].grid(True, linestyle='--', alpha=0.7)

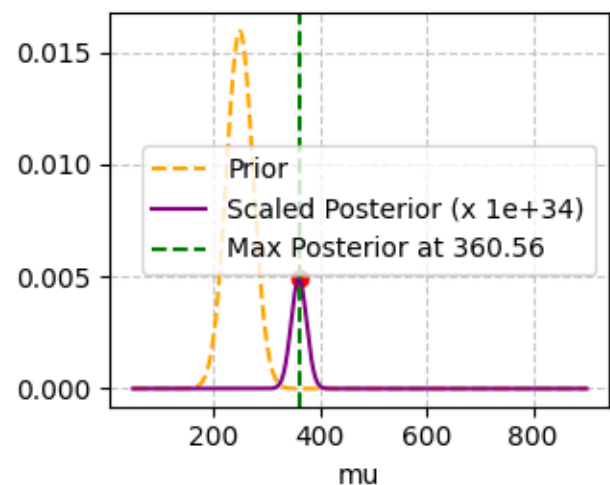
plt.tight_layout()
plt.show()

```

Prior and Unnormalized Posterior



Prior and Scaled Posterior



Part 3: Bayesian Learning

- Data (number of accidents) : $k = 25, 20, 23, 27$
- Prior for day 1 : $\lambda_1 \sim \text{Gamma}(40, 2)$
- The posterior for day i : $\lambda_i \sim \text{Gamma}(\alpha_i, \beta_i)$ is posterior distribution of day $i+1$: $\lambda_{i+1} \sim \text{Gamma}(\alpha_i + k_i, \beta_i + 1)$
- The likelihood function (Poisson Distribution) : $L(\lambda|k) = \frac{\lambda^k e^{-\lambda}}{k!}$

Prior on to generate predictions for day 5

We start with the initial prior which is $\sim \text{Gamma}(40, 2)$.

Day 1:

Observed data $k_1 = 25$.

Posterior distribution after Day 1: $\sim \text{Gamma}(40 + 25, 2 + 1) = \text{Gamma}(65, 3)$.

Day 2:

Observed data $k_2 = 20$.

Posterior distribution after Day 2: $\sim \text{Gamma}(65 + 20, 3 + 1) = \text{Gamma}(85, 4)$.

Day 3:

Observed data $k_3 = 23$.

Posterior distribution after Day 3: $\sim \text{Gamma}(85 + 23, 4 + 1) = \text{Gamma}(108, 5)$.

Day 4:

Observed data $k_4 = 27$.

Posterior distribution after Day 4: $\sim \text{Gamma}(108 + 27, 5 + 1) = \text{Gamma}(135, 6)$

The posterior distribution of after Day 4, $\sim \text{Gamma}(135, 6)$, becomes the prior for predicting the number of accidents on Day 5.

3.1 So, the prior distribution on the 5th day is **Gamma(135, 6)**

The prediction of the model would be the expected value of the random variable k , which is itself since it follows the poisson distribution.

To get the value of λ we can use its expected value to predict for that day. So, the predicted number of accidents on day 5 would be $E[E[k]] = E[\lambda] = \frac{\alpha}{\beta} = 135/6 = \mathbf{22.5}$

3.2 So, the predicted value would be **22** or **23** as it cannot be fractional.

Part 4: Model building in the Bayesian framework

Model Assumptions

Null Model:

- For both $T_w, T_{nw} \sim \mathcal{N}(\mu, \sigma)$ (Because $\delta = 0$)
- $\mu \sim \mathcal{N}(300, 50)$, $\sigma = 60$, $\delta = 0$

Lexical-access Model:

- $T_w \sim \mathcal{N}(\mu, \sigma)$, $T_{nw} \sim \mathcal{N}(\mu + \delta, \sigma)$
- $\mu \sim \mathcal{N}(300, 50)$, $\delta \sim \mathcal{N}_+(0, 50)$, $\sigma = 60$

Generating δ from a truncated normal distribution:

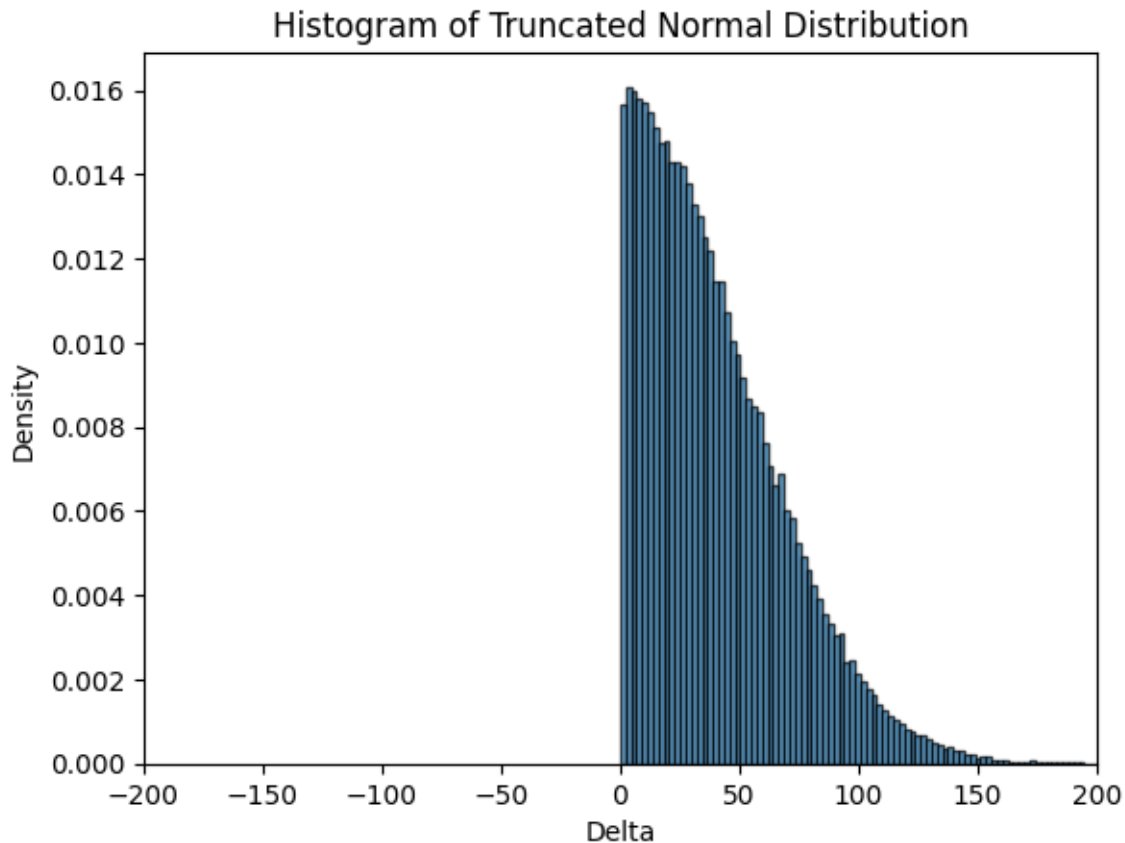
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import truncnorm

num_samples = 100000
lower_bound = 0
upper_bound = np.inf
mean = 0
std_dev = 50

a_norm = (lower_bound - mean) / std_dev
b_norm = (upper_bound - mean) / std_dev

delta = truncnorm.rvs(a=a_norm, b=b_norm, loc=mean, scale=std_dev, size=
    num_samples)

plt.hist(delta, bins=100, density=True, edgecolor='black', alpha=0.7)
plt.xlim(-200, 200)
plt.xlabel('Delta')
plt.ylabel('Density')
plt.title('Histogram of Truncated Normal Distribution')
plt.show()
```



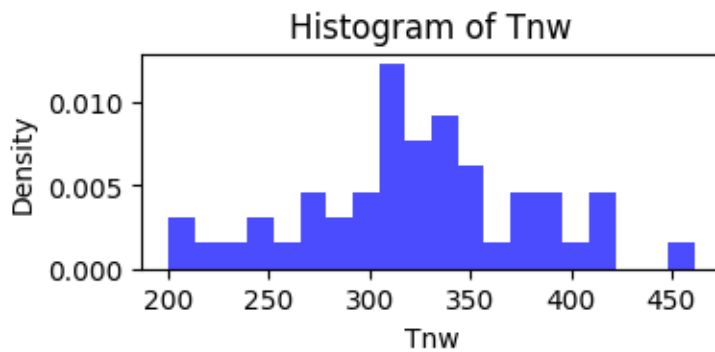
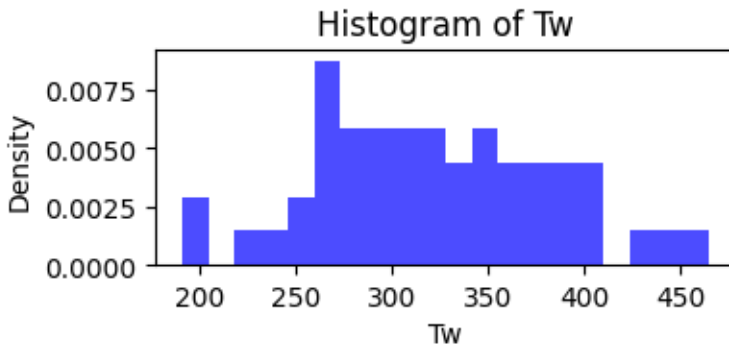
4.4 Data Loading

```
from scipy.stats import norm
import pandas as pd

url = "https://raw.githubusercontent.com/yadavhimanshu059/CGS698C/main/notes/Module-2/recognition.csv"
data = pd.read_csv(url)
Tw = data['Tw'].values
Tnw = data['Tnw'].values

plt.figure(figsize=(4,4))
plt.subplot(2,1,1)
plt.hist(Tw, bins=20, density=True, color='blue', alpha = 0.7)
plt.title('Histogram of Tw')
plt.xlabel('Tw')
plt.ylabel('Density')
plt.subplot(2,1,2)
plt.hist(Tnw, bins=20, density=True, color='blue', alpha = 0.7)
plt.title('Histogram of Tnw')
plt.xlabel('Tnw')
```

```
plt.ylabel('Density')
plt.tight_layout()
plt.show()
```



4.5.1 Unnormalised Posterior under Null:

```
def like_est(mu, sig, delt):
    return (np.prod(norm.pdf(data["Tw"], mu, sig)) * np.prod(norm.pdf(data["
        Tnw"], mu + delt, sig)))

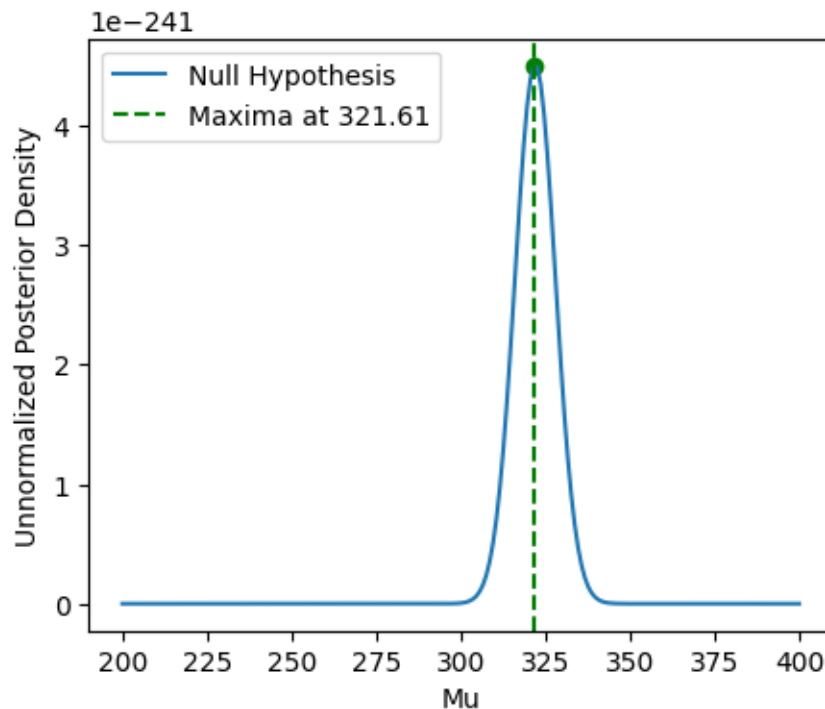
x_lab = np.linspace(200, 400, 200)
sig = 60.0
delt = 0.0

y_post_null = [(norm.pdf(x, 300, 50) * like_est(x, sig, delt)) for x in
    x_lab]

plt.figure(figsize=(5, 4))
plt.plot(x_lab, y_post_null, label='Null_Hypothesis', linestyle='--')
max_idx = np.argmax(y_post_null)
max_mu = x_lab[max_idx]
max_post = y_post_null[max_idx]
plt.axvline(max_mu, color='g', linestyle='--', label=f'Maxima_at_{max_mu:.2f
}')

```

```
plt.scatter(max_mu, max_post, color='g')
plt.xlabel('Mu')
plt.ylabel('Unnormalized Posterior Density')
plt.legend()
plt.show()
```



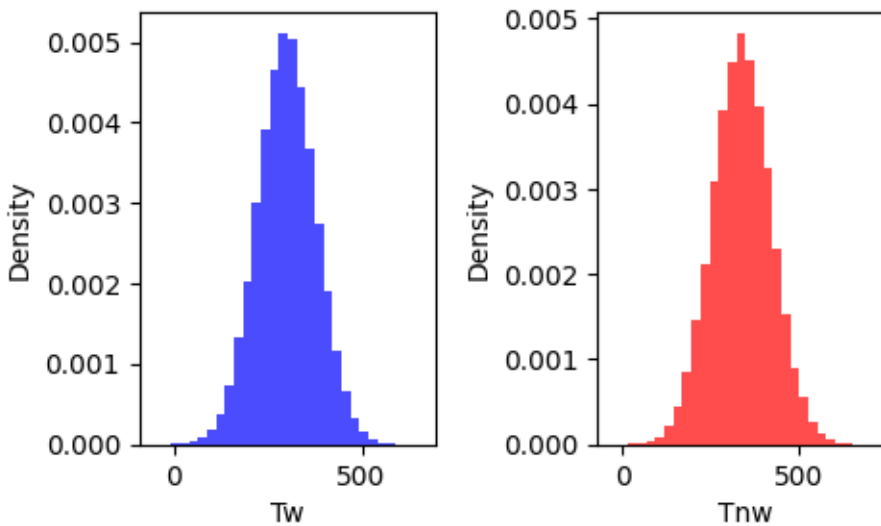
0.1 4.5.2 Generate the prior predictions from the lexical-access model:

```
m_mean = 300
m_std = 50
d_mean = 0
d_std = 50
s = 60
n = 100000

m_samples = np.random.normal(m_mean, m_std, n)
d_samples = truncnorm.rvs(a=0, b=np.inf, loc=d_mean, scale=d_std, size=n)
Tw_samples = [np.random.normal(m, s, 1)[0] for m in m_samples]
Tnw_samples = [np.random.normal(m + d, s, 1)[0] for m, d in zip(m_samples,
    d_samples)]

plt.figure(figsize=(5, 3))
plt.subplot(1, 2, 1)
```

```
plt.hist(Tw_samples, bins=30, alpha=0.7, color='blue', density=True)
plt.xlabel('Tw')
plt.ylabel('Density')
plt.subplot(1, 2, 2)
plt.hist(Tnw_samples, bins=30, alpha=0.7, color='red', density=True)
plt.xlabel('Tnw')
plt.ylabel('Density')
plt.tight_layout()
plt.show()
```



0.2 4.5.3 Compare the prior predictions of the null hypothesis model and the lexical access model:

Since for Null model we have $\delta = 0$, priors for both word recognition times and non-word recognition times are the same.

Comparing the Null and Lexical Access model, we find that the priors for word recognition are the same in both of the cases.

In the case of non-word recognition we have,

```
mw = np.mean(Tw_samples)
mdw = np.median(Tw_samples)
mnw = np.mean(Tnw_samples)
mdnw = np.median(Tnw_samples)
```

```
mu_mean = 300
mu_std = 50
sigma = 60
delta = delta
n = 100000
```

```

mu_samples = np.random.normal(mu_mean, mu_std, n)
Tlw_samples = [np.random.normal(mu, sigma) for mu in mu_samples]
Tlnw_samples = [np.random.normal(mu + delta, sigma) for mu in mu_samples]

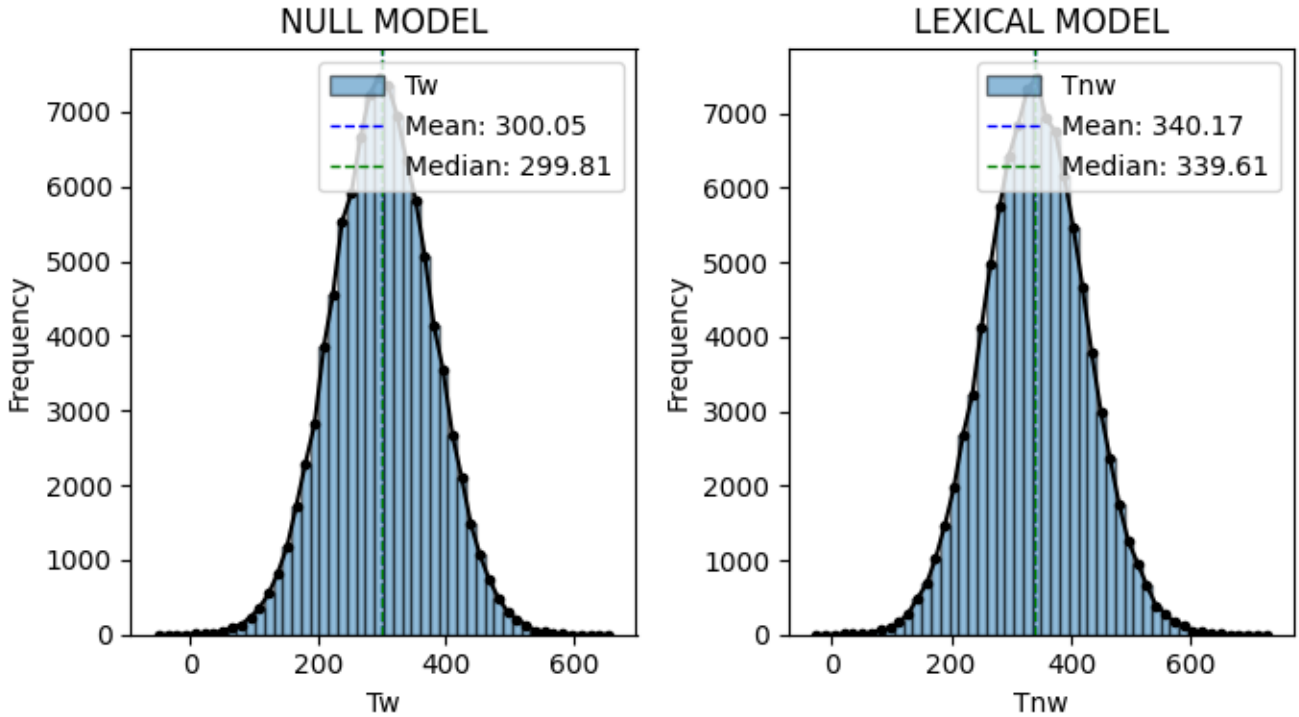
mlw = np.mean(Tlw_samples)
mldw = np.median(Tlw_samples)
mlnw = np.mean(Tlnw_samples)
mldnw = np.median(Tlnw_samples)

fig, axs = plt.subplots(nrows=1, ncols=2, figsize=(7, 4))

cts, bns, patches = axs[0].hist(Tnw_samples, bins=50, alpha=0.5, label='Tnw',
    , edgecolor='black')
bc = 0.5 * (bns[:-1] + bns[1:])
axs[0].plot(bc, cts, linestyle='-', marker='.', color='k')
axs[0].axvline(mnw, color='blue', linestyle='dashed', linewidth=1, label=f'
    Mean: {mnw:.2f}')
axs[0].axvline(mdw, color='green', linestyle='dashed', linewidth=1, label=f'
    Median: {mdnw:.2f}')
axs[0].set_xlabel('Tnw')
axs[0].set_ylabel('Frequency')
axs[0].set_title('NULL MODEL')
axs[0].legend()

cts, bns, patches = axs[1].hist(Tlnw_samples, bins=50, alpha=0.5, label='Tnw',
    , edgecolor='black')
bc = 0.5 * (bns[:-1] + bns[1:])
axs[1].plot(bc, cts, linestyle='-', marker='.', color='k')
axs[1].axvline(mnw, color='blue', linestyle='dashed', linewidth=1, label=f'
    Mean: {mlnw:.2f}')
axs[1].axvline(mdnw, color='green', linestyle='dashed', linewidth=1, label=f'
    Median: {mldnw:.2f}')
axs[1].set_xlabel('Tnw')
axs[1].set_title('LEXICAL MODEL')
axs[1].legend()
plt.tight_layout()
plt.show()

```



NULL MODEL Prior: Mean = 299.71, Median = 299.81 LEXICAL MODEL Prior: Mean = 299.83, Median = 299.75 Comparison conclusion: Lexical Hypothesis suggests longer non-word recognition times compared to Null Hypothesis.

0.3 4.5.4 Model more consistent with the data

```
fig, axs = plt.subplots(nrows=3, ncols=2, figsize=(6, 8))

cts, bns, patches = axs[1, 0].hist(Tw_samples, bins=50, alpha=0.5, label='Tw',
                                     edgecolor='black')
axs[1, 0].axvline(mw, color='blue', linestyle='dashed', linewidth=1, label=f'Mean: {mw:.2f}')
axs[1, 0].axvline(mdw, color='green', linestyle='dashed', linewidth=1, label=f'Median: {mdw:.2f}')
axs[1, 0].set_xlabel('Tw')
axs[1, 0].set_ylabel('Frequency')
axs[1, 0].set_title('NULL MODEL')
axs[1, 0].legend()

cts, bns, patches = axs[0, 0].hist(data["Tw"], bins=50, alpha=0.5, label='Given Data',
                                     edgecolor='black')
axs[0, 0].axvline(np.mean(data["Tw"]), color='blue', linestyle='dashed',
                  linewidth=1, label=f'Mean: {np.mean(data["Tw"]):.2f}')
```

```

axs[0, 0].axvline(np.median(data["Tw"]), color='green', linestyle='dashed',
    linewidth=1, label=f'Median: {np.median(data["Tw"]):.2f}')
axs[0, 0].set_xlabel('Tw')
axs[0, 0].set_ylabel('Frequency')
axs[0, 0].set_title('Given Data')
axs[0, 0].legend()

```

```

cts, bns, patches = axs[1, 1].hist(Tnw_samples, bins=50, alpha=0.5, label='
    Tnw', edgecolor='black')
axs[1, 1].axvline(mnw, color='blue', linestyle='dashed', linewidth=1, label=
    f'Mean: {mnw:.2f}')
axs[1, 1].axvline(mdnw, color='green', linestyle='dashed', linewidth=1,
    label=f'Median: {mdnw:.2f}')
axs[1, 1].set_xlabel('Tnw')
axs[1, 1].set_ylabel('Frequency')
axs[1, 1].set_title('NULL MODEL')
axs[1, 1].legend()

```

```

cts, bns, patches = axs[0, 1].hist(data["Tnw"], bins=50, alpha=0.5, label='
    Given Data', edgecolor='black')
axs[0, 1].axvline(np.mean(data["Tnw"]), color='blue', linestyle='dashed',
    linewidth=1, label=f'Mean: {np.mean(data["Tnw"]):.2f}')
axs[0, 1].axvline(np.median(data["Tnw"]), color='green', linestyle='dashed',
    linewidth=1, label=f'Median: {np.median(data["Tnw"]):.2f}')
axs[0, 1].set_xlabel('Tnw')
axs[0, 1].set_ylabel('Frequency')
axs[0, 1].set_title('Given Data')
axs[0, 1].legend()

```

```

cts, bns, patches = axs[2, 1].hist(Tlnw_samples, bins=50, alpha=0.5, label='
    Tnw', edgecolor='black')
axs[2, 1].axvline(mnw, color='blue', linestyle='dashed', linewidth=1, label=
    f'Mean: {mlnw:.2f}')
axs[2, 1].axvline(mdnw, color='green', linestyle='dashed', linewidth=1,
    label=f'Median: {mldnw:.2f}')
axs[2, 1].set_xlabel('Tnw')
axs[2, 1].set_ylabel('Frequency')
axs[2, 1].set_title('LEXICAN MODEL')
axs[2, 1].legend()

```

```

cts, bns, patches = axs[2, 0].hist(Tw_samples, bins=50, alpha=0.5, label='Tw
    ', edgecolor='black')
axs[2, 0].axvline(mw, color='blue', linestyle='dashed', linewidth=1, label=f

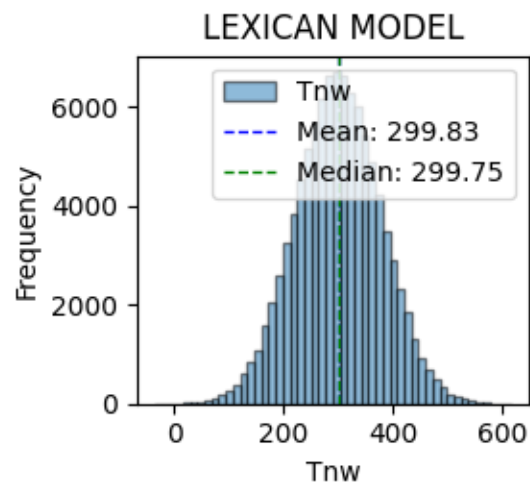
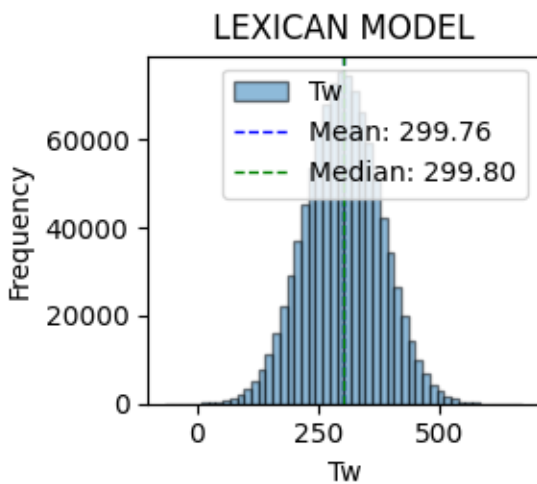
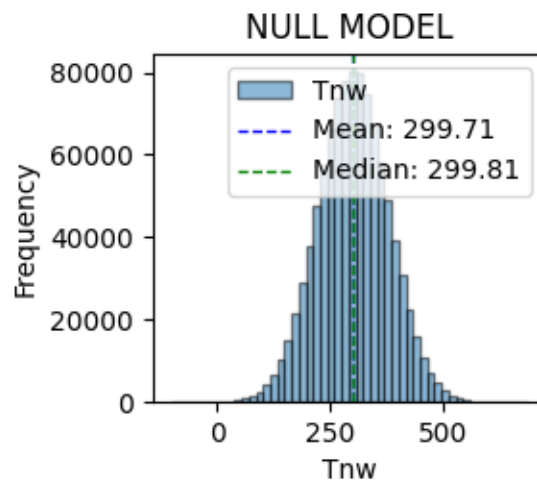
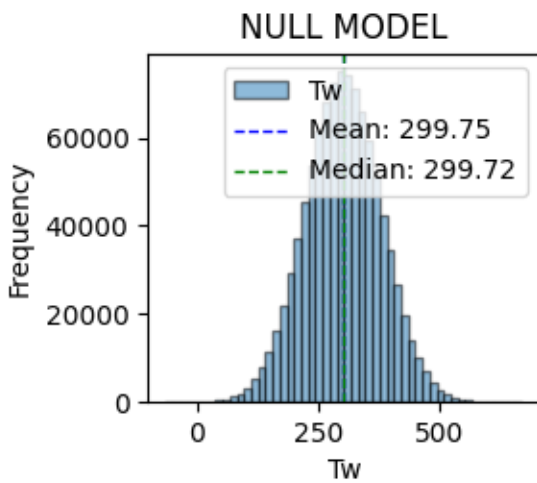
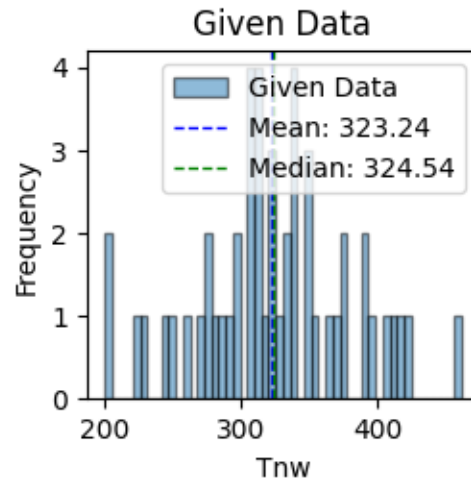
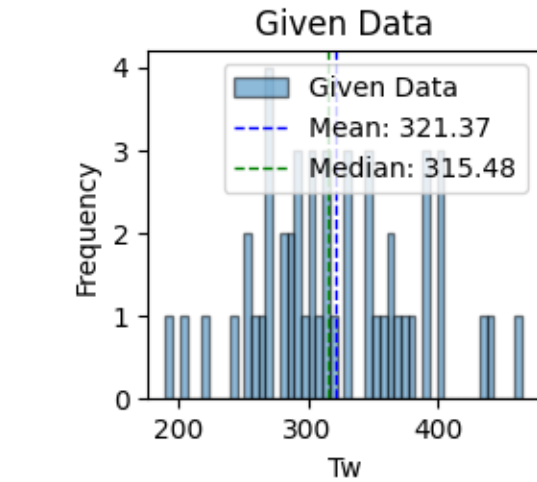
```

```

    'Mean:_{mlw:.2f}'))
    axs[2, 0].axvline(mdw, color='green', linestyle='dashed', linewidth=1, label
        =f'Median:_{mldw:.2f}')
    axs[2, 0].set_xlabel('Tw')
    axs[2, 0].set_ylabel('Frequency')
    axs[2, 0].set_title('LEXICAN_MODEL')
    axs[2, 0].legend()

plt.tight_layout()
plt.show()

```



0.4 4.5.5

```
import emcee
```

```

def log_prob(params, Tw, Tnw):
    m, d = params

    if d < 0:
        return -np.inf

    mp = -((m - 300)**2) / (2 * 50**2)
    dp = -((d - 0)**2) / (2 * 50**2)
    Twl = -0.5 * np.sum(((Tw - m)**2) / 60**2 + np.log(2 * np.pi * 60**2))
    Tnw1 = -0.5 * np.sum(((Tnw - (m + d))**2) / 60**2 + np.log(2 * np.pi *
        60**2))

    return mp + dp + Twl + Tnw1

ini = [300, 50]
nd = 2
nw = 50
ns = 2000
nb = 1000
p = np.array(ini) + 1e-4 * np.random.randn(nw, nd)
sampler = emcee.EnsembleSampler(nw, nd, log_prob, args=(Tw, Tnw))
sampler.run_mcmc(p, ns, progress=True)
s = sampler.get_chain(discard=nb, flat=True)
ds = s[:, 1]
d_label = np.linspace(0, 100, 100)
mu_samples = np.random.normal(300, 50, 500)
posterior_distribution = []

fig, axes = plt.subplots(nrows=1, ncols=1, figsize=(6, 4))
for mu_val in mu_samples:
    y_posterior = [np.exp(log_prob([mu_val, delta_val], Tw, Tnw)) * np.abs(
        norm.pdf(delta_val, 0, 50)) for delta_val in d_label]
    axes.plot(d_label, y_posterior, linestyle='-', color='b', alpha=0.1)
    posterior_distribution.append(y_posterior)

posterior_distribution = np.array(posterior_distribution)
mean_posterior = np.mean(posterior_distribution, axis=0)
max_index = np.argmax(mean_posterior)
max_delta = d_label[max_index]
max_posterior = mean_posterior[max_index]

axes.plot(d_label, mean_posterior, label='Posterior_distribution_for_Delta',
    linestyle='-', color='r')
axes.axvline(max_delta, color='k', linestyle='--', label=f'Max_posterior_at_
    {max_delta:.2f}')

```

```

axes.scatter(max_delta, max_posterior, color='b')
axes.set_xlabel('Delta')
axes.set_ylabel('Unnormalized Posterior Density')
axes.set_title('Posterior Distribution of Delta')
axes.legend()

```

```

plt.tight_layout()
plt.show()

```

